

INFORME DE INCIDENTE · PIPELINE INTRALE

Diez horas en vacío

Por qué el 8 de septiembre el pipeline consumió cuota todo el día, relanzó agentes sin sentido y produjo un solo PR.

2026-09-08 Ventana analizada: 00:00Z → 01:00Z (+1)

Fuentes: spawn-exit · cross-provider-dispatch · pulpo.log · liveness

No hubo una falla del modelo ni de los agentes. El pipeline pasó ~10 horas intentando lanzar agentes con la cadena de providers completamente apagada, a razón de un intento cada 21 segundos, sin lograr lanzar ni uno. Ese giro en vacío hizo que el brazo de huérfanos contara esas horas como “el agente murió”, devolviera las fases a la cola y quemara sus 3 reintentos hasta rechazarlas. Todo lo que se hizo de día fue rehacer lo que la noche había tirado.

EL DÍA EN NÚMEROS

Mucho movimiento, casi nada de avance

7538 eventos de dispatch	51 llegaron a lanzar un agente (0,7%)	1 PR mergeado a <code>main</code>	912 salidas de agente auditadas (2,2× el día anterior)
928 mensajes a Telegram	58% cuota semanal Anthropic <i>sin usar</i> al cierre		

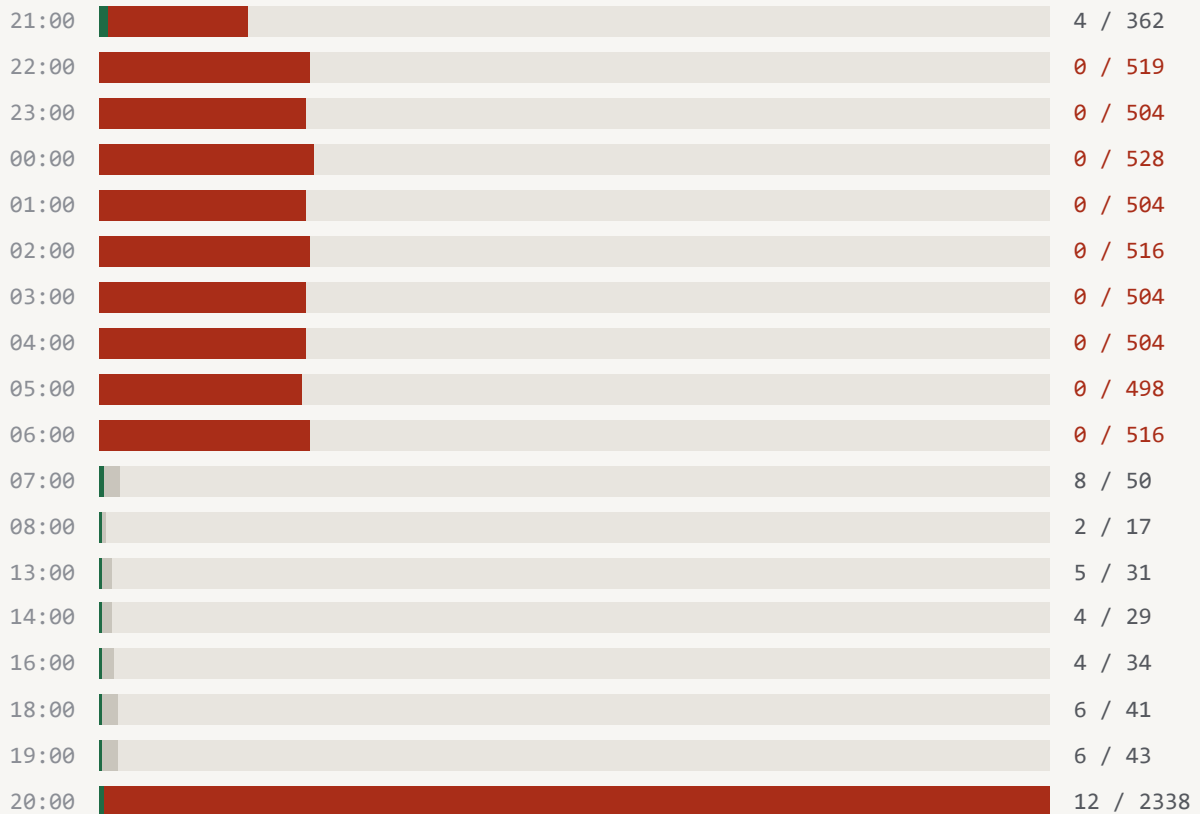
El contador de ola marca 93% de avance y el watchdog de dispatch atribuye el estancamiento a `causeKind: "quota"`. Ninguna de las dos cosas describe lo que

pasó: la cuota estaba disponible y el avance real fueron 9 PR el sábado, 1 el domingo y 1 el lunes.

RECONSTRUCCIÓN HORARIA

Dónde se fue el día

Cada barra es una hora. El largo es la cantidad de intentos de dispatch; el segmento verde, los que efectivamente lanzaron un agente. Hora de Buenos Aires.



■ giro en vacío (cadena de providers agotada) ■ operación normal ■ agentes efectivamente lanzados

Las horas 09:00–12:00, 15:00, 17:00 y 21:00 no registraron eventos de dispatch. La barra de las 20:00 es de esta misma noche: el ciclo **ya arrancó de nuevo** — 2338 intentos en una hora, 12 lanzados.

MECANISMO

Cómo un apagón nocturno se convirtió en trabajo rechazado

1 Anthropic entra en su ventana de reposo a las 20:00

```
Es
política
declarada,
no
un
bug:
provider-
schedule.json
apaga
Anthropic
de
20:00
a
07:00.
Verificado
en
vivo:
isProviderActiveNow('anthropic')
===
false .
```

2 La cadena de respaldo no tiene a nadie de pie

OpenAI/Codex
bloqueado
por
flag
de
cuota
(2190
veces),
Gemini
sin
licencia
de
CLI
(453),
Kimi
sin
credenciales
(86),

Cerebras
 agotado,
 NVIDIA
 NIM
 caído.
 La
 cadena
 anthropic
 →
 openai-
 codex
 se
 agotó
 1719
 veces.

3 El Pulpo reintenta cada 21 segundos, toda la noche

Al
 agotarse
 la
 cadena
 no
 hay
 backoff
 ni
 espera
 hasta
 la
 próxima
 ventana
 activa:
 vuelve
 a
 intentar
 de
 inmediato.
 ~170
 chain_exhausted
 por
 hora,

diez
horas
seguidas.

4 El marcador de fase queda en **trabajando/** sin proceso detrás

El
brazo
de
huérfanos
mide
"minutos
sin
proceso",
no
"minutos
sin
provider
disponible".
Contó
las
horas
de
apagón
como
si el
agente
se
hubiera
muerto.

5 Devuelve la fase a la cola y quema los reintentos

Tres
pasadas
y la
fase
queda
rechazada.
Les

pasó
a las
dos
piezas
centrales
de la
ola,
por
una
causa
que
no
tiene
nada
que
ver
con
su
código.

6 De día se rehace desde cero

De
ahí
los
attempt-
2 en
guru,
po,
ux y
pipeline-
dev
sobre
los
mismos
7
issues,
y los
~113
M

de
tokens
(40%
del
día)
gastados
en
repetir
trabajo
ya
hecho.

```
[huerfanos] 5801.pipeline-dev lleva 638min sin proceso → pendiente/
(intento 3/3)
[huerfanos] 5801.pipeline-dev excedió 3 reintentos → rechazado
[huerfanos] 6239.pipeline-dev lleva 629min sin proceso → pendiente/
(intento 3/3)
[huerfanos] 6239.pipeline-dev excedió 3 reintentos → rechazado
```

638 minutos = 10 h 38 min. Exactamente la ventana de reposo más el arranque.

FALLAS DE FONDO

Las seis cosas que hay que arreglar

BLOQUEANTE

El dispatch no frena cuando no hay a quién despachar

Con toda la cadena apagada, el Pulpo reintentará en bucle cerrado en lugar de dormir hasta la próxima ventana activa. Es el generador del 95% del ruido del día y el disparador de todo lo que sigue.

```
chain_exhausted: 2143    reason: "all_gated"
fallback_also_gated: 2672    primary_inactive_by_schedule: 2133
fallback_selected: 51
```

.pipeline/lib/agent-launcher/dispatch-with-fallback.js — emisión de `chain_exhausted`

BLOQUEANTE

El brazo de huérfanos castiga al issue por una falla del proveedor

Ya existe el precedente correcto: cuando el spawn falla por el provider se registra un marker y *no* se penaliza el retry del issue. Falta extender ese mismo criterio al caso “nunca se intentó porque no había provider”. Sin esto, cada noche de apagón se come tres reintentos de cada fase en vuelo.

`.pipeline/lib/agent-launcher/spawn-failure-state.js` — el mecanismo a extender

BLOQUEANTE

Un provider bloqueado por cuota no puede desbloquearse solo

El flag de cuota se limpia únicamente cuando ese provider tiene un spawn exitoso — pero mientras el flag está puesto, el provider no puede spawnear. Es un candado circular: OpenAI/Codex bloqueó 2190 dispatches con el flag puesto, y hoy su cuota real marca 0% de uso. Cerebras está marcado agotado hasta mañana a las 23:00Z por una detección de las 20:00.

```
quota-exhausted.json → cerebras: exhausted:true, resets_at 2026-09-09T23:00:29Z
multi-provider-health → openai: quota pct 10, state green
```

`.pipeline/lib/quota-exhausted.js` — `clearFlag` sólo en `success_spawn`

ALTA

La telemetría de diagnóstico está envenenada

El clasificador marca como error transitorio *toda* corrida que dure más de 30 segundos y no traiga un error reconocible — es decir, todos los agentes sanos, que corren entre 5 y 27 minutos. Resultado: 809 de 912 salidas auditadas como `transient_5xx`, 682 de ellas con `exit_code: 0` y muerte normal. Verificado que **no dispara relanzamientos**: el Pulpo decide el éxito por `exit_code`. Pero convierte en basura la única fuente que sirve para diagnosticar incidentes de provider — que es parte de por qué no aparecía una causa clara.

```
if (timedOut === true || durationMs >= 30000) return { errorClass: 'transient_5xx' }
```

`.pipeline/lib/agent-launcher/provider-error-parser.js:814` — `classifyByContext`

ALTA

El diagnóstico automático apunta al lugar equivocado

El watchdog de dispatch reporta `causeKind: "quota"` con Anthropic al 58% semanal y 76% de sesión disponibles. La causa real es de agenda y disponibilidad

de cadena, no de cuota. Mientras el sistema se autodiagnostique mal, cada incidente arranca la investigación por la pista falsa.

MEDIA

El ruido de Telegram tapa la señal

928 mensajes en el día, de los cuales 177 son alertas del propio servicio de Telegram sobre sí mismo. Se emitieron además 2112 eventos de pausa por agotamiento — uno por cada vuelta del bucle — y varios fallaron al enviarse. Hay que agrupar por episodio, no por evento.

DESCARTADO CON EVIDENCIA

Lo que no fue

- **No fue cuota agotada.** Anthropic cerró el día con 58% semanal y 76% de sesión disponibles.
- **No fue calidad del código ni rebotes de QA.** Un solo rebote en todo el día, y de infraestructura. #5801 y #6239 tienen `qa:passed`.
- **No fue el watchdog matando al Pulpo.** Hubo un solo `kill-respawn` en todo el día, a las 21:45 — consecuencia del final del día, no su causa.
- **No fue el modelo ni los agentes.** Cuando lograron correr, terminaron con salida normal; el problema fue que su trabajo se descartaba después.

PLAN DE SALIDA

Qué hacer, en este orden

Fase 0 · Frenar el sangrado

esta noche

1. Parar el pipeline hasta las 07:00.

El ciclo de las 20:00 ya empezó: 2338 intentos en una hora. No hay ningún provider capaz de trabajar de noche, así que cada hora que siga corriendo sólo quema reintentos de fases sanas. Halt total con `.paused`.

2. Rescatar los tres marcadores en vuelo antes de que respawnee.

5113.pipeline-dev , 5691.ux y 6239.guru están en trabajando/ sin proceso detrás. Si el Pulpo levanta con el brazo de huérfanos activo, se los come igual que anoche.

3. Levantar el Pulpo, que está caído.

Fue matado por el watchdog a las 21:45 y el respawn no prendió; el pid 21116 ya no existe. Desde PowerShell, nunca desde Git Bash.

```
node .pipeline/restart.js
```

4. Devolver #5801 y #6239 a la cola sin penalización.

Fueron rechazados por infraestructura, no por su código. Reponer sus contadores de reintento en cero.

Fase 1 · Cortar la causa raíz

mañana, antes de reabrir la ola

1. Backoff en el dispatch cuando la cadena se agota.

Ante all_gated , dormir hasta la próxima ventana en que algún provider vuelva a estar activo, en lugar de reintentar cada 21 segundos. Solo esto elimina ~5000 eventos por noche.

2. El tiempo sin provider no cuenta como tiempo sin proceso.

El brazo de huérfanos debe descontar los minutos en los que ningún provider estuvo disponible. Una fase que nunca pudo lanzarse no es huérfana y no debe gastar reintentos.

3. Expiración real del flag de cuota por provider.

Que el flag caduque por resets_at y se revalide contra el adaptador de cuota, sin depender de un spawn exitoso que el propio flag impide.

Fase 2 · Recuperar la capacidad de diagnosticar

esta semana

1. Decidir qué pasa de noche, explícitamente.

Hoy hay una contradicción de política: se apaga Anthropic 11 horas y se deja al pipeline intentando igual, con una cadena de respaldo que en la práctica no existe. Las dos salidas

honestas son conseguir un respaldo nocturno que funcione de verdad, o declarar la noche como parada del pipeline. La mezcla actual es la que produce el daño.

2. Arreglar `classifyByContext`.

Que "duró más de 30 segundos" deje de significar error. Exigir `timedOut` o salida distinta de cero. Sin esto, el próximo incidente vuelve a investigarse a ciegas.

3. Corregir la atribución de causa del watchdog de stall.

Si la cuota está disponible, la causa no es cuota. Distinguir agenda, disponibilidad de cadena y cuota como causas separadas.

4. Agrupar las notificaciones por episodio.

Un aviso por episodio de agotamiento, no uno por vuelta del bucle. Y sacar del canal las alertas que el servicio de Telegram se manda a sí mismo.

VERIFICACIÓN

Cómo sabemos que salimos

Cuatro números, medibles al día siguiente sobre los mismos archivos que se usaron para este informe.

SEÑAL	HOY	OBJETIVO
Eventos de dispatch por día	7538	< 500
Tasa de dispatch efectivo	0,7%	> 60%
Fases rechazadas por huérfano	3	0
Corridas <code>attempt ≥ 2</code>	40% del gasto	< 10%

Si mañana a la mañana el conteo de `chain_exhausted` de la noche no bajó a cero, la Fase 1 no quedó aplicada — es la verificación más rápida que hay.

Elaborado el 2026-09-08 sobre spawn-exit-2026-09-0*.jsonl, cross-provider-dispatch-2026-09-08.jsonl, pulpo.log, pulpo-liveness.log, service-liveness.log, svc-telegram.log y el estado del kanban en .pipeline/desarrollo/.